FlowPilot AI

UML design: Identity, Organizations, Roles & Permissions, Audit and AI integration

Item	Value
Owner	Soufiane (initial codebase creator)
Modules covered	`identity`, `organizations`, `audit`, `ai`, `shared.security`, and the matching frontend parts
Contents	Module map, 2 use-case diagrams, 9 class diagrams, 8 sequence diagrams, permission matrices, REST endpoints, frontend structure
Purpose	Tell the other teams the exact class, method and endpoint names they will call, and the interfaces they must implement for the AI analyst
Version / date	1.0 / 2026-11-04 (design before implementation: Phase 2.2 onwards)

How to read this document

- **Colours**: blue = Soufiane's modules, orange = interfaces or classes another team implements, green = shared kernel, purple = framework or external system (Spring, Ollama).
- **Arrows**: solid line with open arrow = uses / has a reference; dashed open arrow = depends on, returns or creates; dashed line with hollow triangle = implements; filled diamond = composition (part of).
- **Packages**: everything lives in `com.flowpilot.<module>`. Only the module's root package is public (the interfaces and records marked "public API"). Classes in `internal.*` must not be used by other modules; a Spring Modulith test fails the build if they are.
- **Sequence diagrams**: messages are numbered; solid arrows are calls, dashed arrows are returns. `alt` = alternatives, `loop` = repetition.

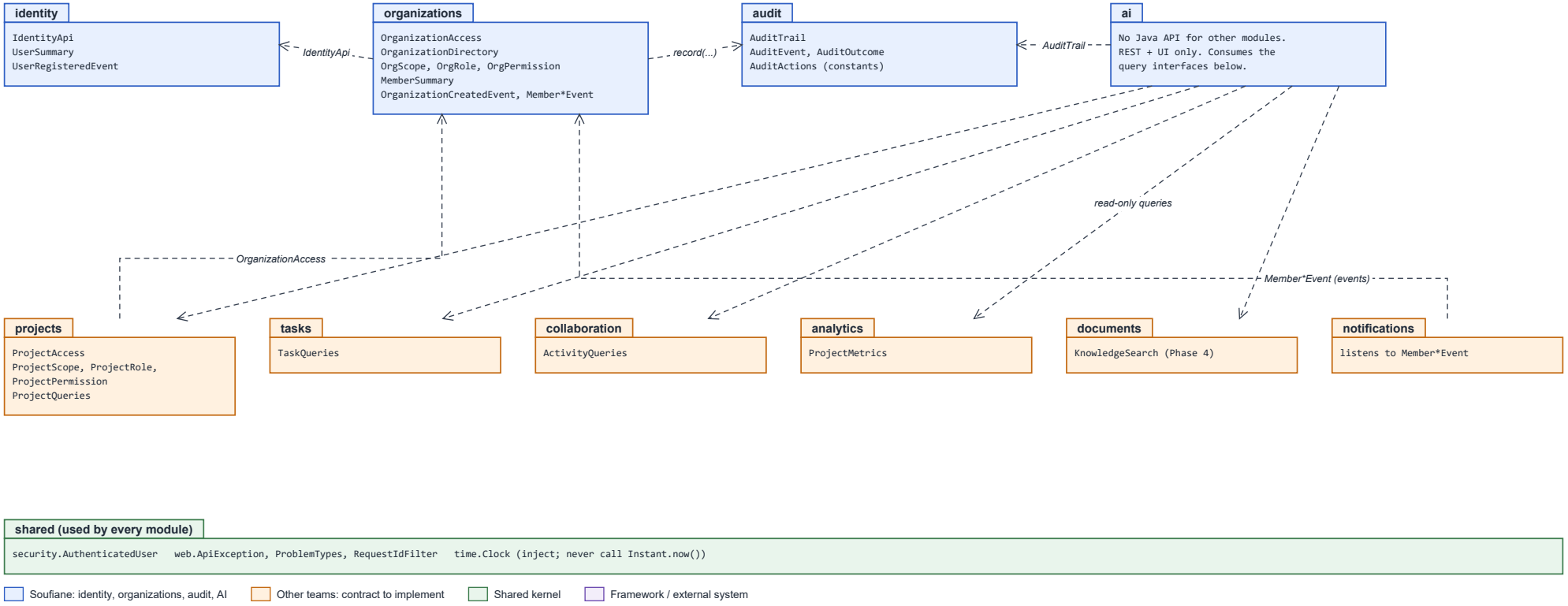
0 Rules every team follows

These rules are what keep tenants isolated and the AI's answers verifiable. They apply to every module, not only to Soufiane's.

- **Resolve access first.** Every endpoint under `/api/v1/orgs/{orgId}/...` starts with `OrganizationAccess.requireMembership/requirePermission` or, for project data, `ProjectAccess.require(orgId, projectId, userId, permission)`. Never trust an `orgId` or `projectId` sent in a request body.
- **Pass the scope, not ids.** Service and query methods on project data take `ProjectScope` (or `OrgScope`) as their first parameter, and every SQL query filters by `scope.orgId()` AND `scope.projectId()`.
- **404, not 403, across tenants.** If the user is not a member of the organization, answer 404 (`ApiException.notFound(...)`) so ids cannot be probed. 403 is only for members missing a permission.
- **The caller** is always `@AuthenticationPrincipal AuthenticatedUser user`. Roles are not in the JWT; they are checked on every request.
- **Return DTO records, never JPA entities.** Records the AI may cite carry a stable `ref` (e.g. `FP-42`, `FP-M3`) that the UI also shows.
- **AI query interfaces are read-only** (`@Transactional(readOnly = true)`), return at most `limit` items (hard cap 50) and should answer in under 200 ms.
- **Audit sensitive actions** with `AuditTrail.record(...)` (see the audit diagram). Never put passwords, tokens, API keys or document text in audit details or logs.
- **Listen to membership events.** The projects team removes project memberships on `MemberRemovedEvent`; the notifications team reacts to `MemberJoinedEvent`.
- **Time** comes from the injected `Clock`; errors are `ApiException` subclasses, rendered as RFC 9457 problem responses by the shared handler.
- **Names here are the contract.** To rename or change a signature, open a pull request that updates this document first.

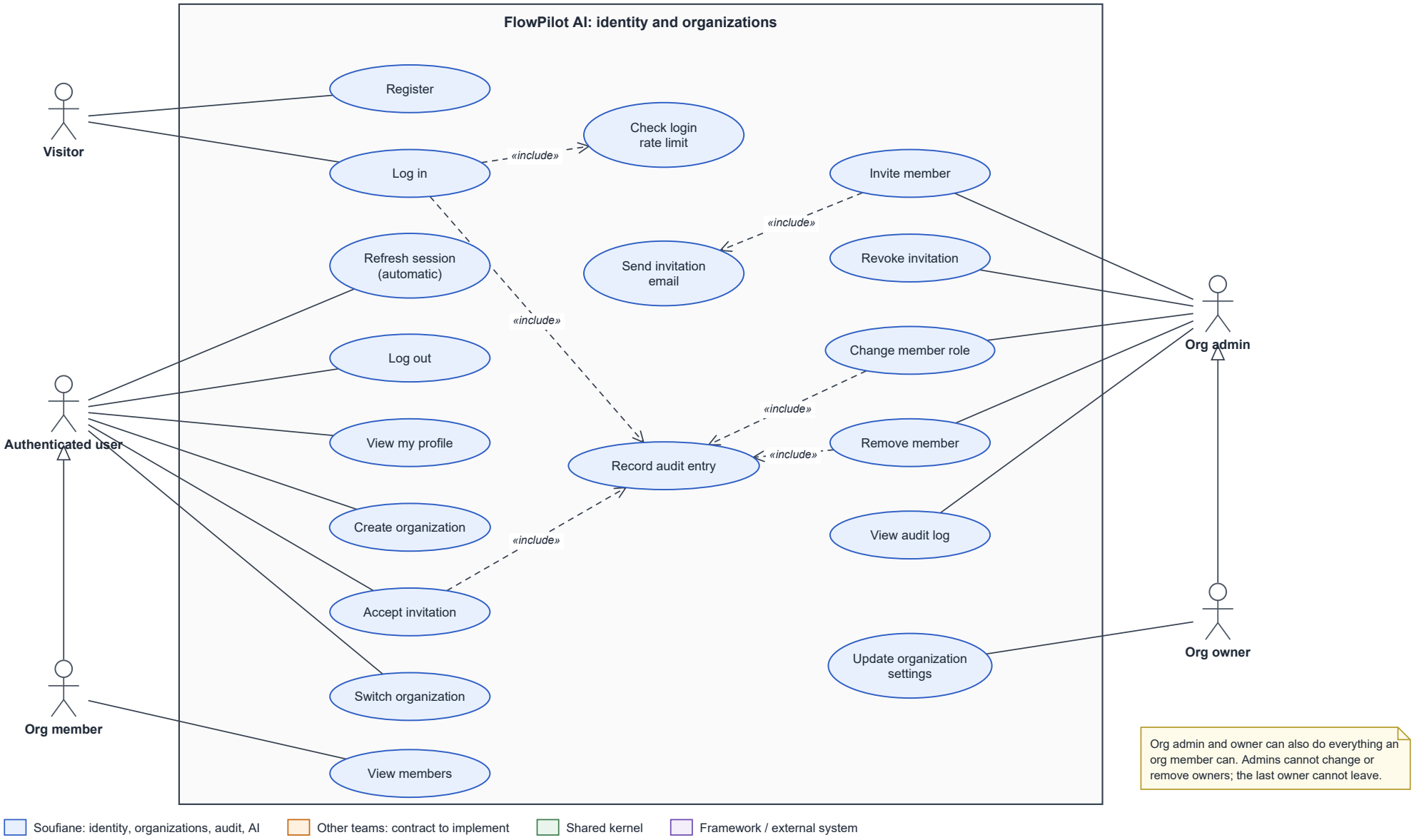
1 Module map and ownership

Who owns which module and which public interfaces each module exposes. Arrows point from the module that uses an interface to the module that provides it.



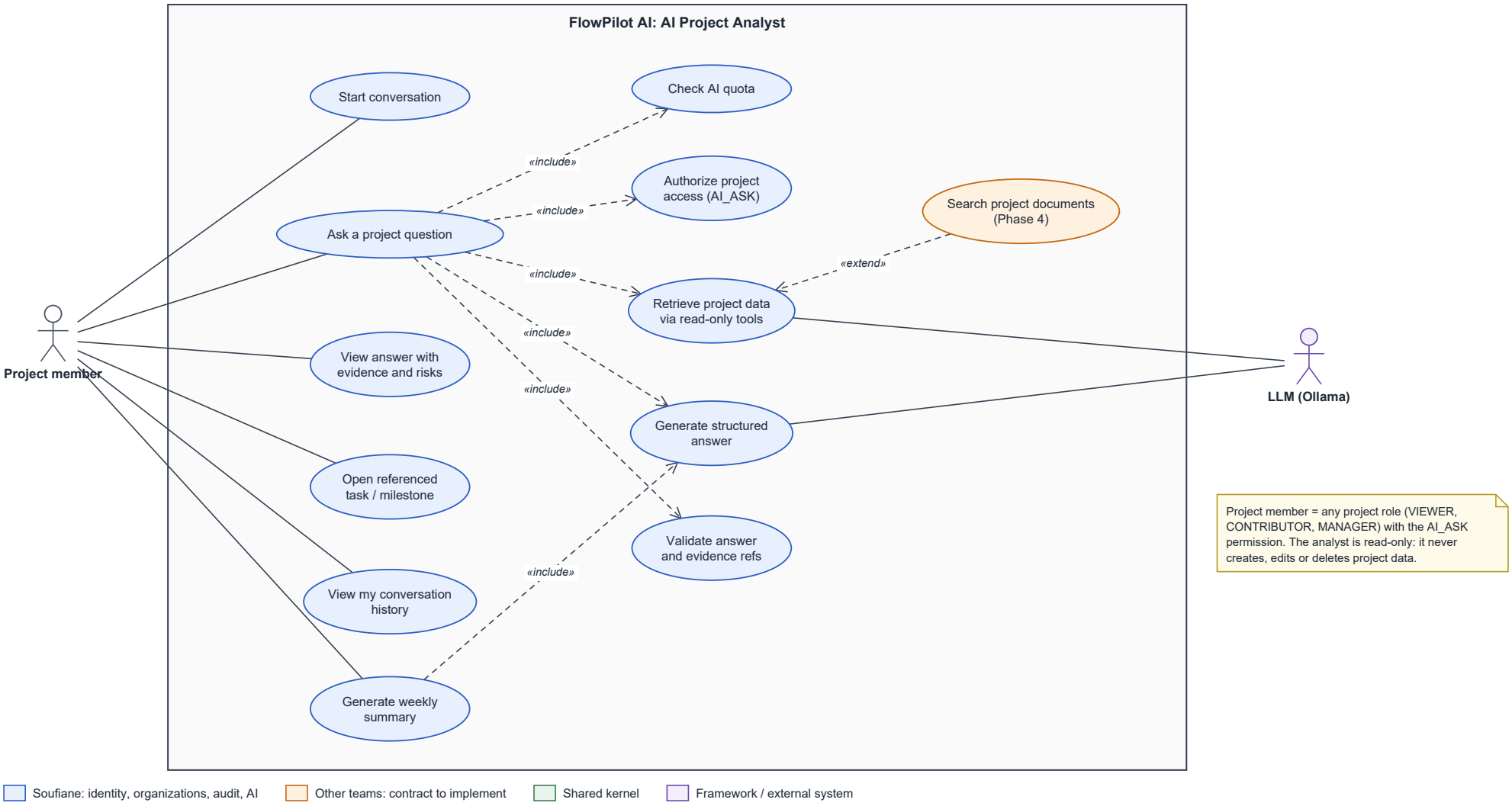
2 Use cases: identity and organizations

What visitors, users, members, admins and owners can do. Sensitive use cases include recording an audit entry.



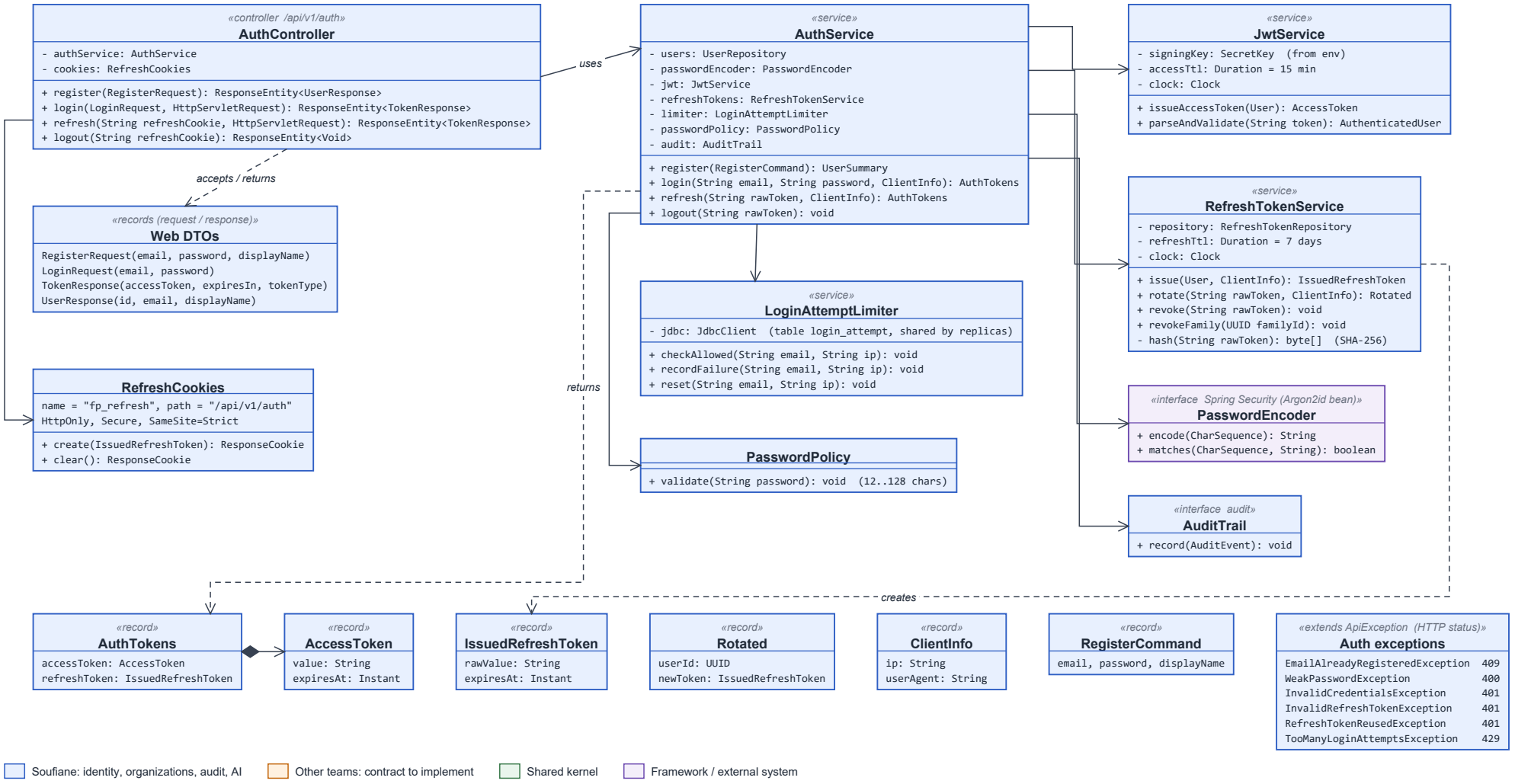
3 Use cases: AI Project Analyst

Asking a question always includes the quota check, the permission check, read-only data retrieval through tools, answer generation and evidence validation.



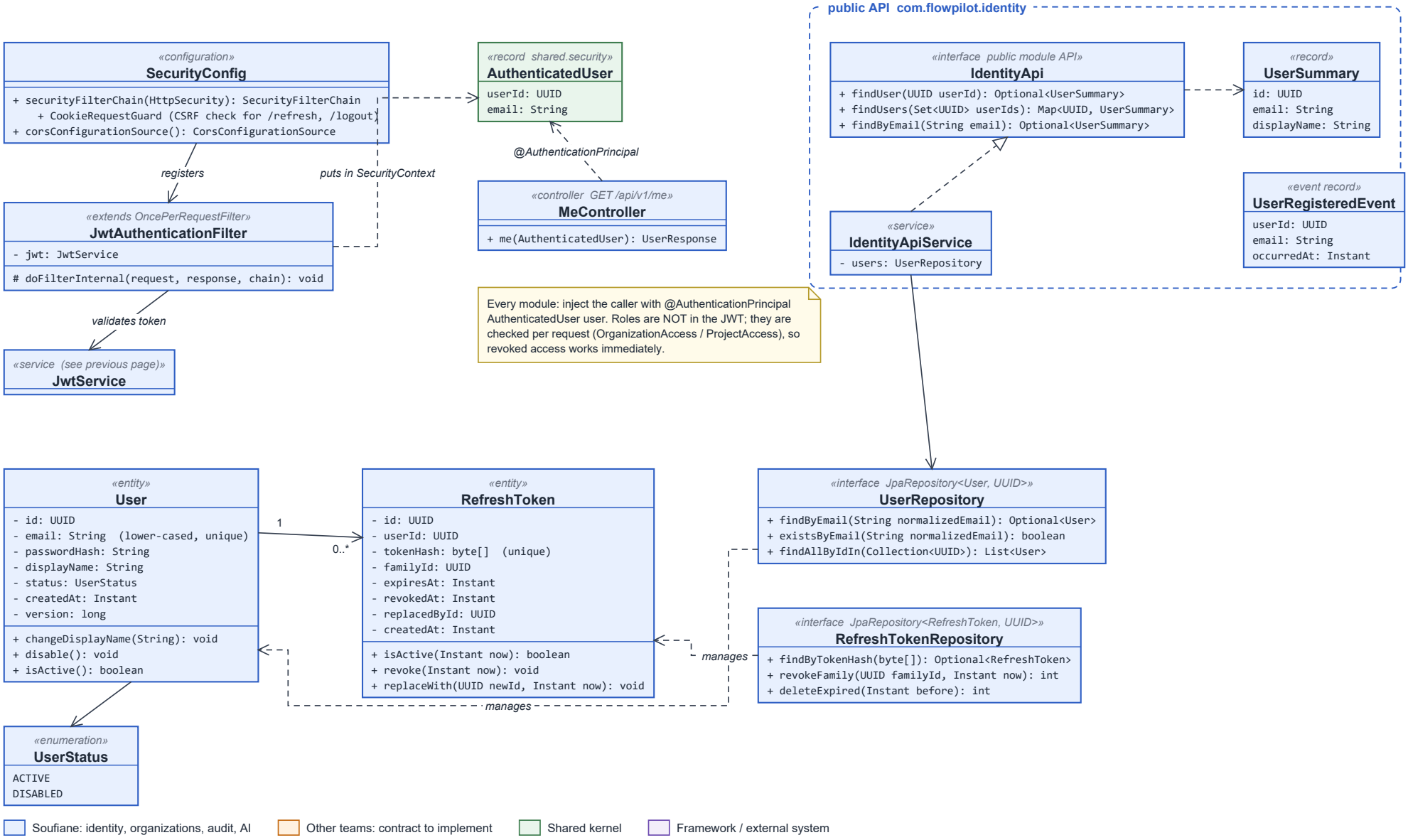
4 Class diagram: authentication and tokens (identity)

Registration, login, refresh-token rotation and logout. Access tokens are short-lived JWTs; refresh tokens are random values stored only as SHA-256 hashes and rotated on every use.



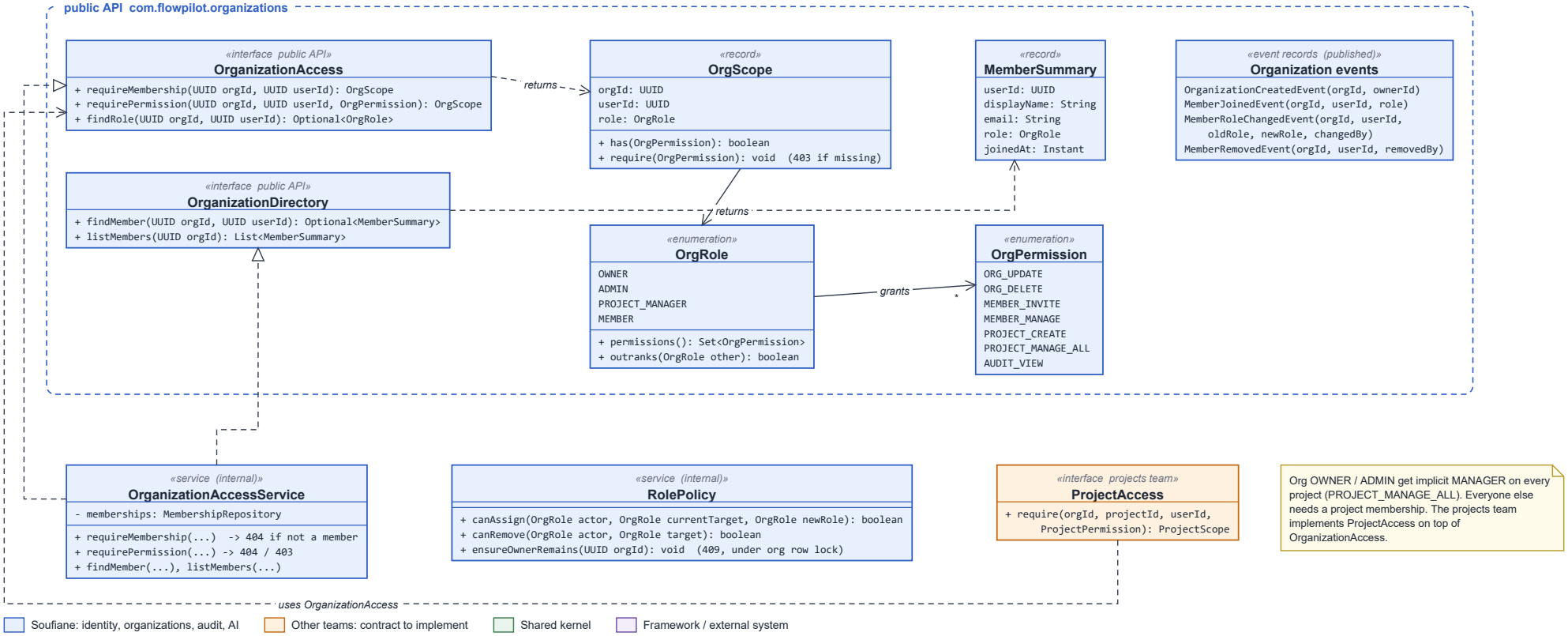
5 Class diagram: security filter, domain, persistence and public API (identity)

How a request becomes an AuthenticatedUser, the entities and repositories, and the IdentityApi other modules may call.



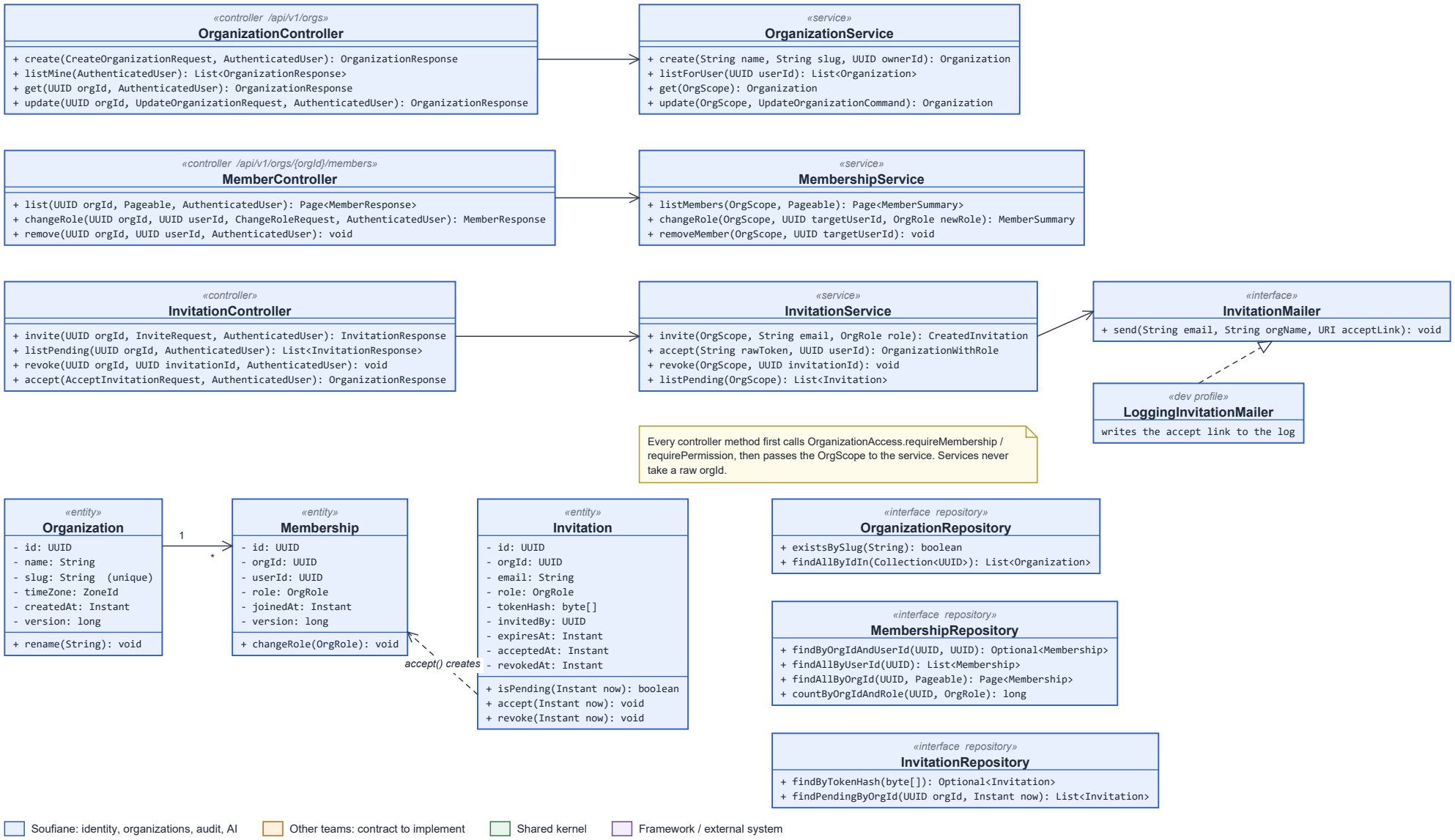
6 Class diagram: organizations public API and RBAC

The interfaces and types other modules use to check organization access and roles. OrganizationAccess is the single place where organization membership is decided.



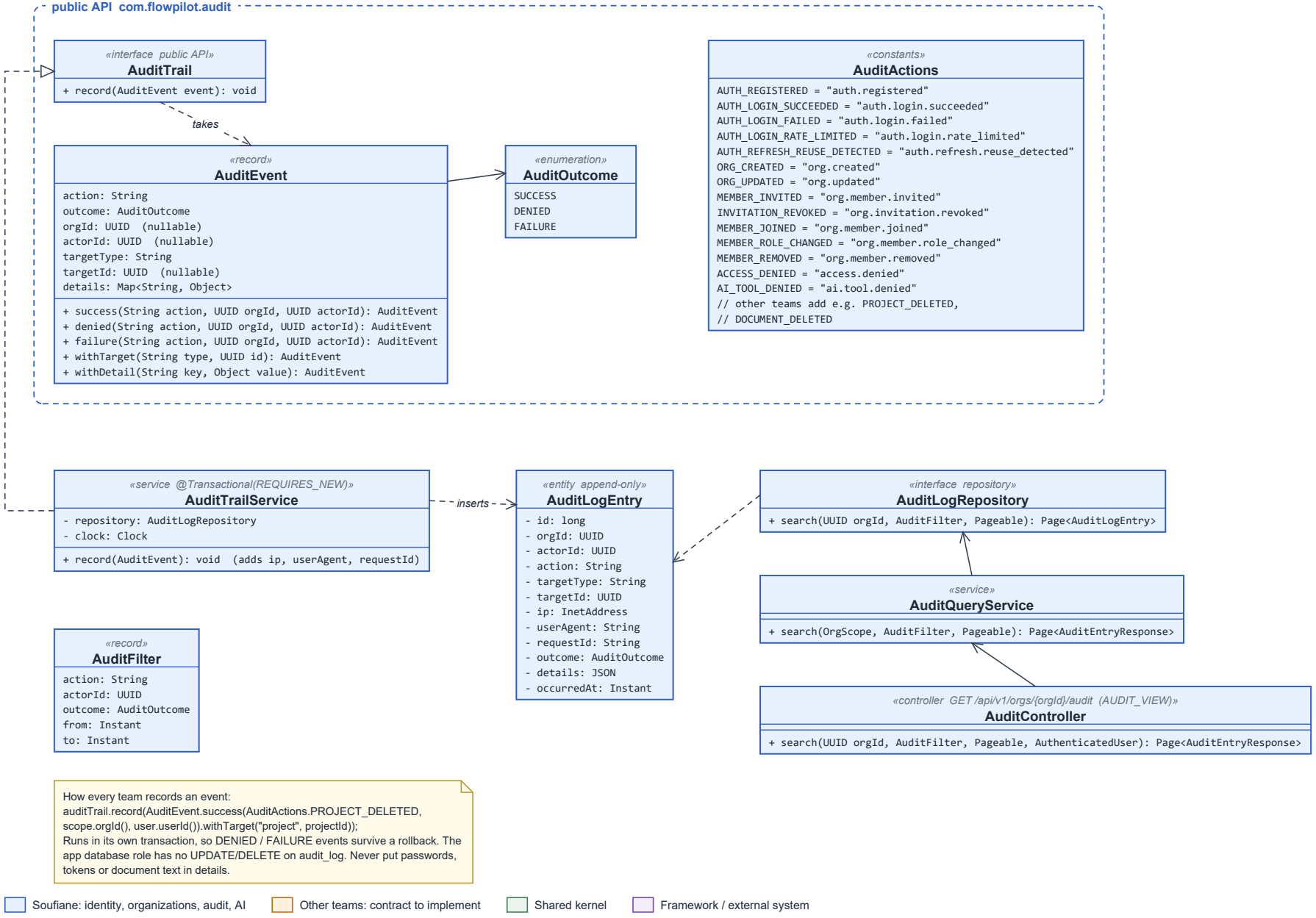
7 Class diagram: organizations internals

Controllers, services, entities and repositories for organizations, members and invitations (internal: not for use by other modules).



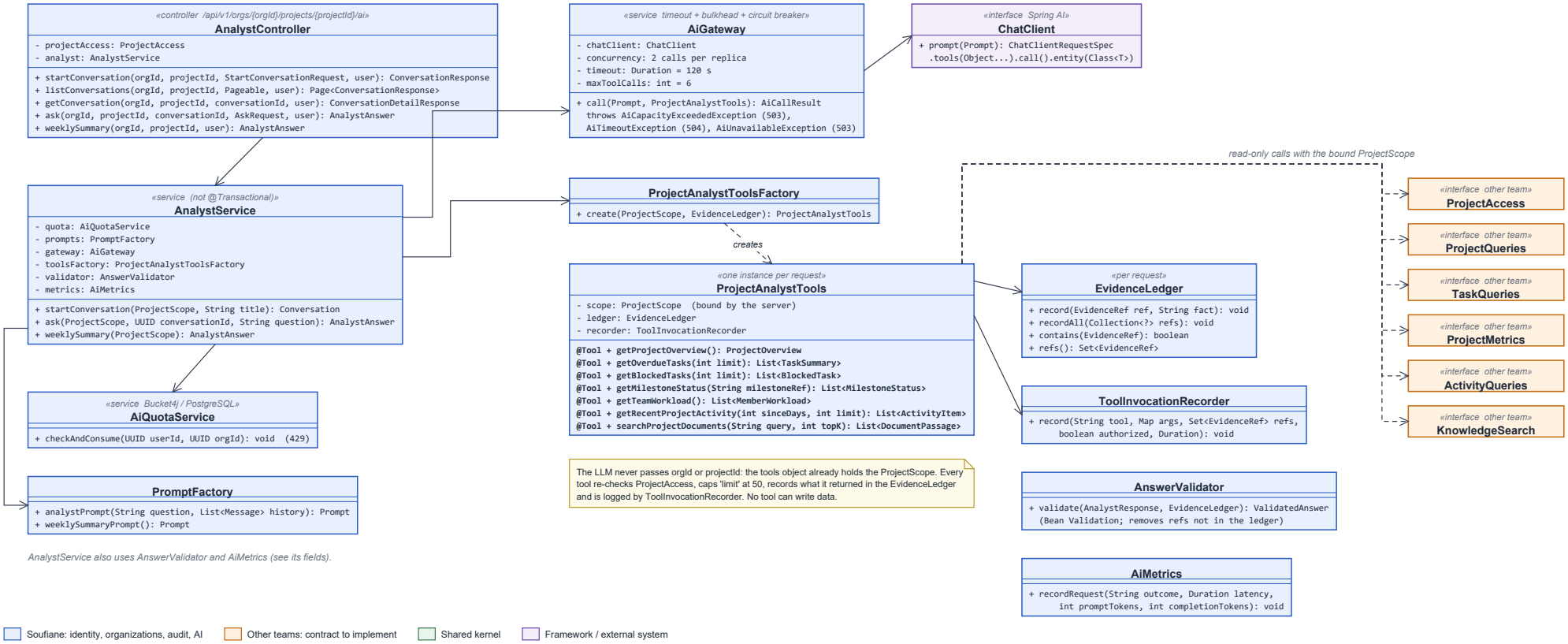
8 Class diagram: audit

A single synchronous API, AuditTrail.record(AuditEvent), used by every module for security-relevant operations, plus the org-scoped audit log query.



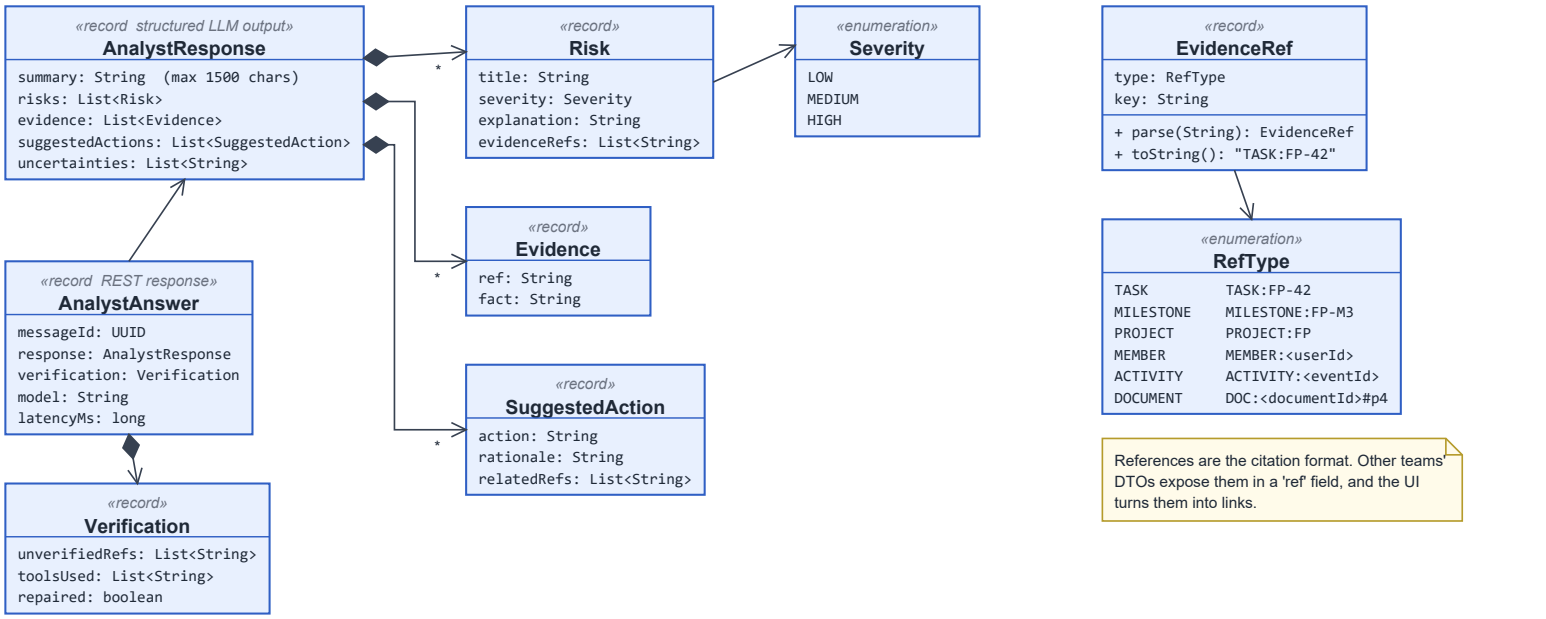
9 Class diagram: AI analyst orchestration and tools

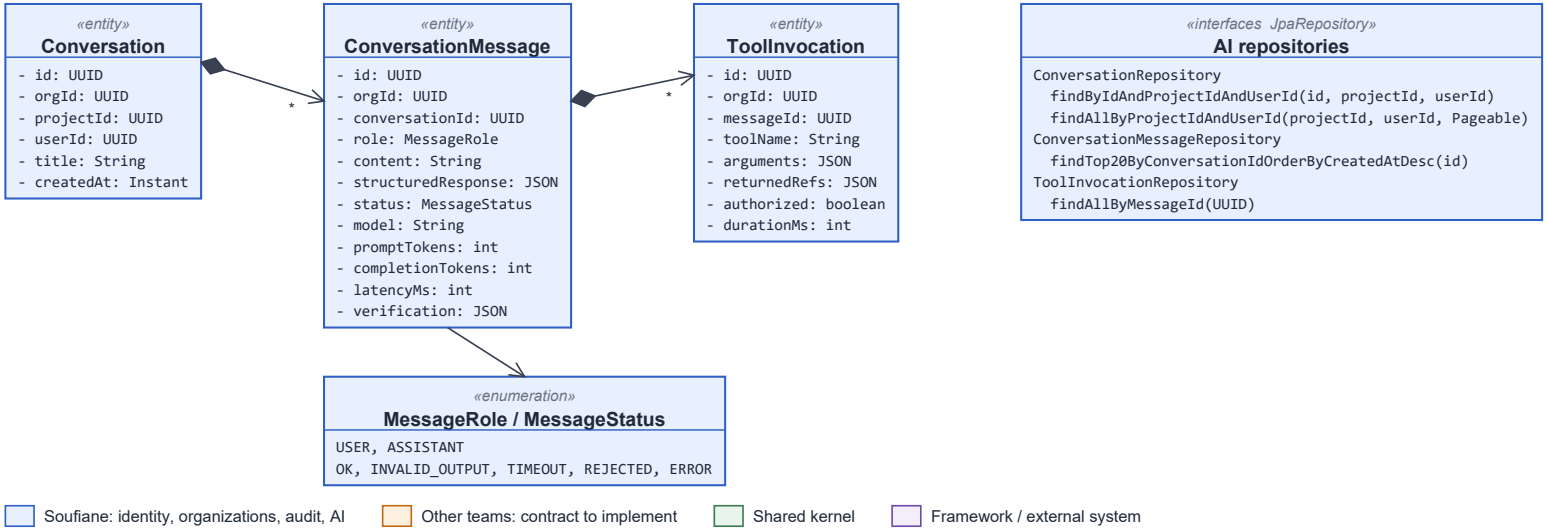
How a question becomes a validated, evidence-backed answer. The orange interfaces are implemented by the other teams (details in sections 11 and 12).



10 Class diagram: AI response model and persistence

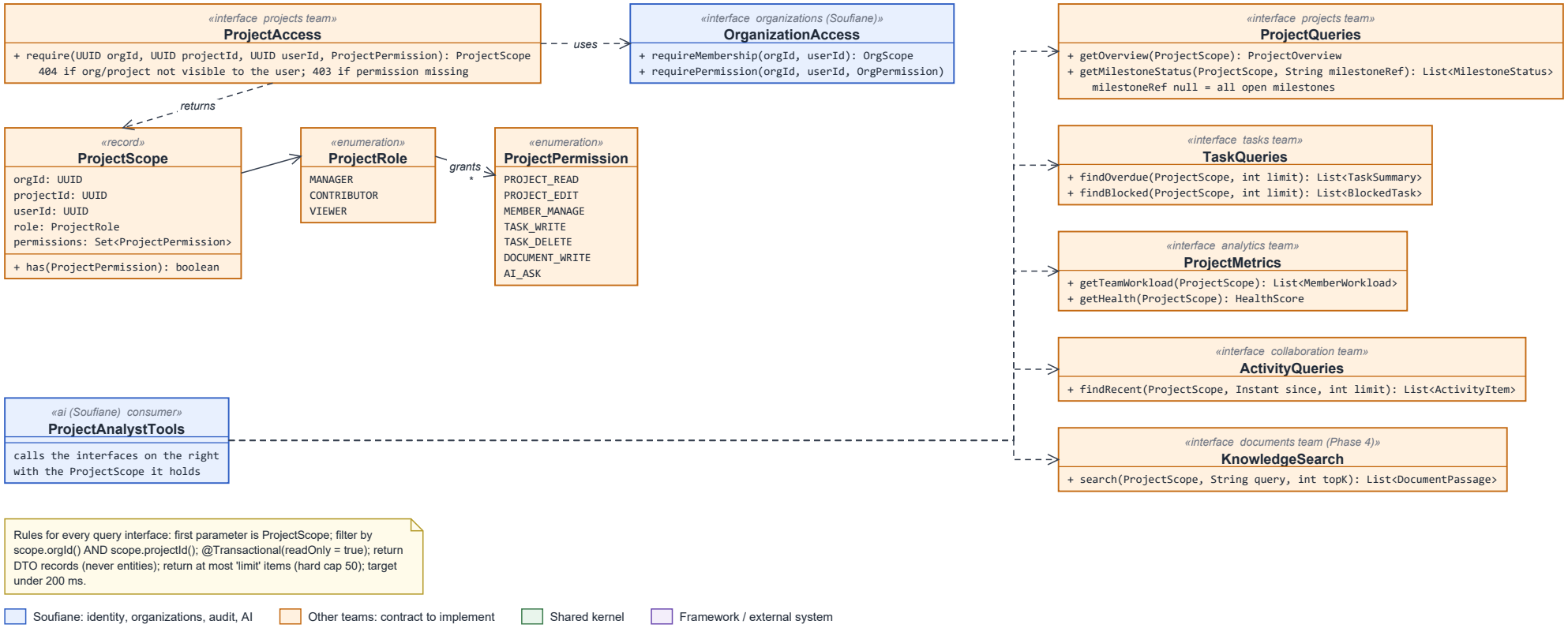
The structured answer the LLM must return, the REST response, the citation format and the stored conversations.





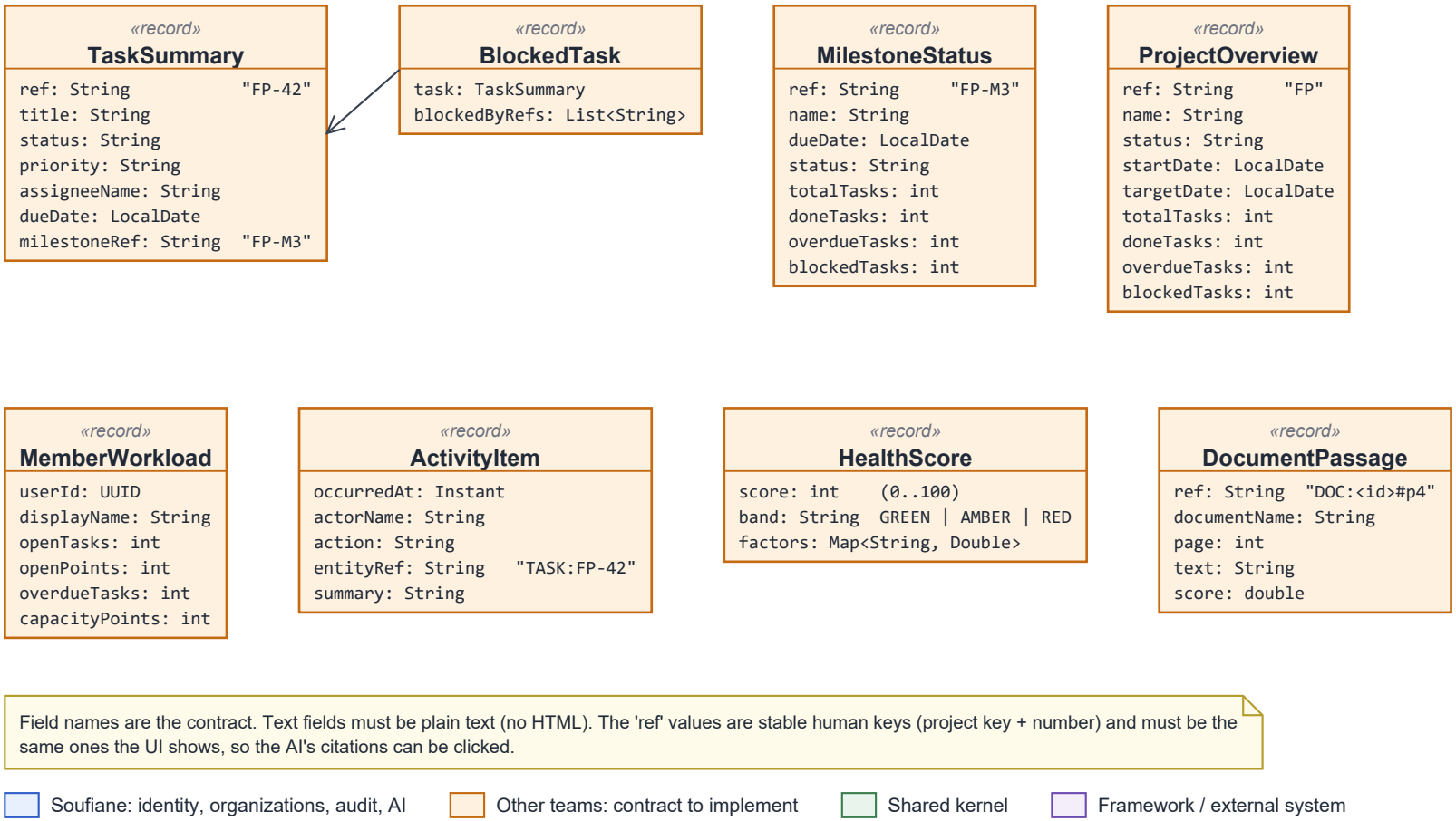
11 Class diagram: interfaces other teams implement for access control and the AI

These are the contracts the AI analyst depends on. Each team implements its interface in the root package of its module.



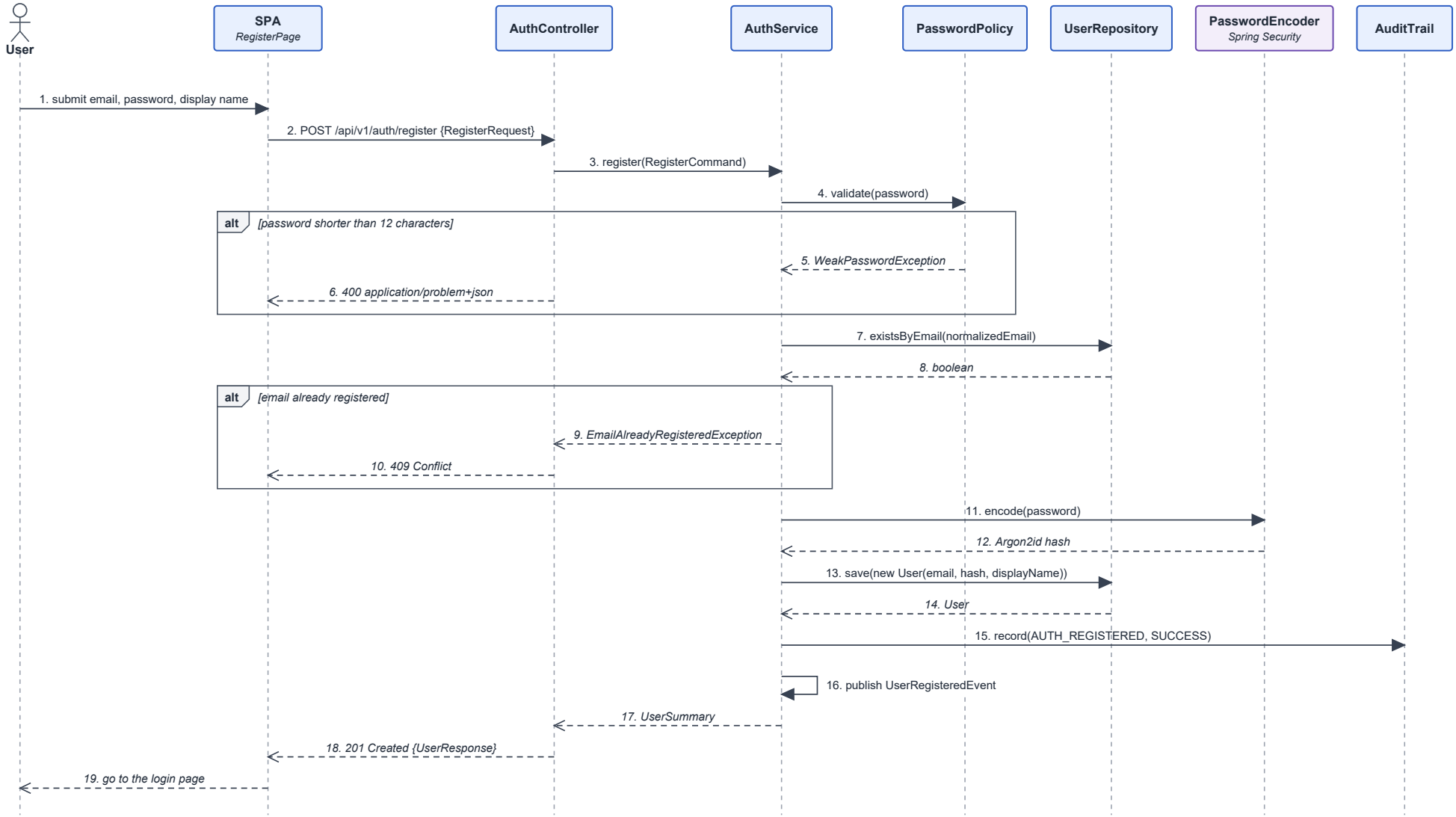
12 Class diagram: DTO records returned by the AI query interfaces

Implemented by the other teams. The AI reads these fields, and the UI uses the refs to build links.



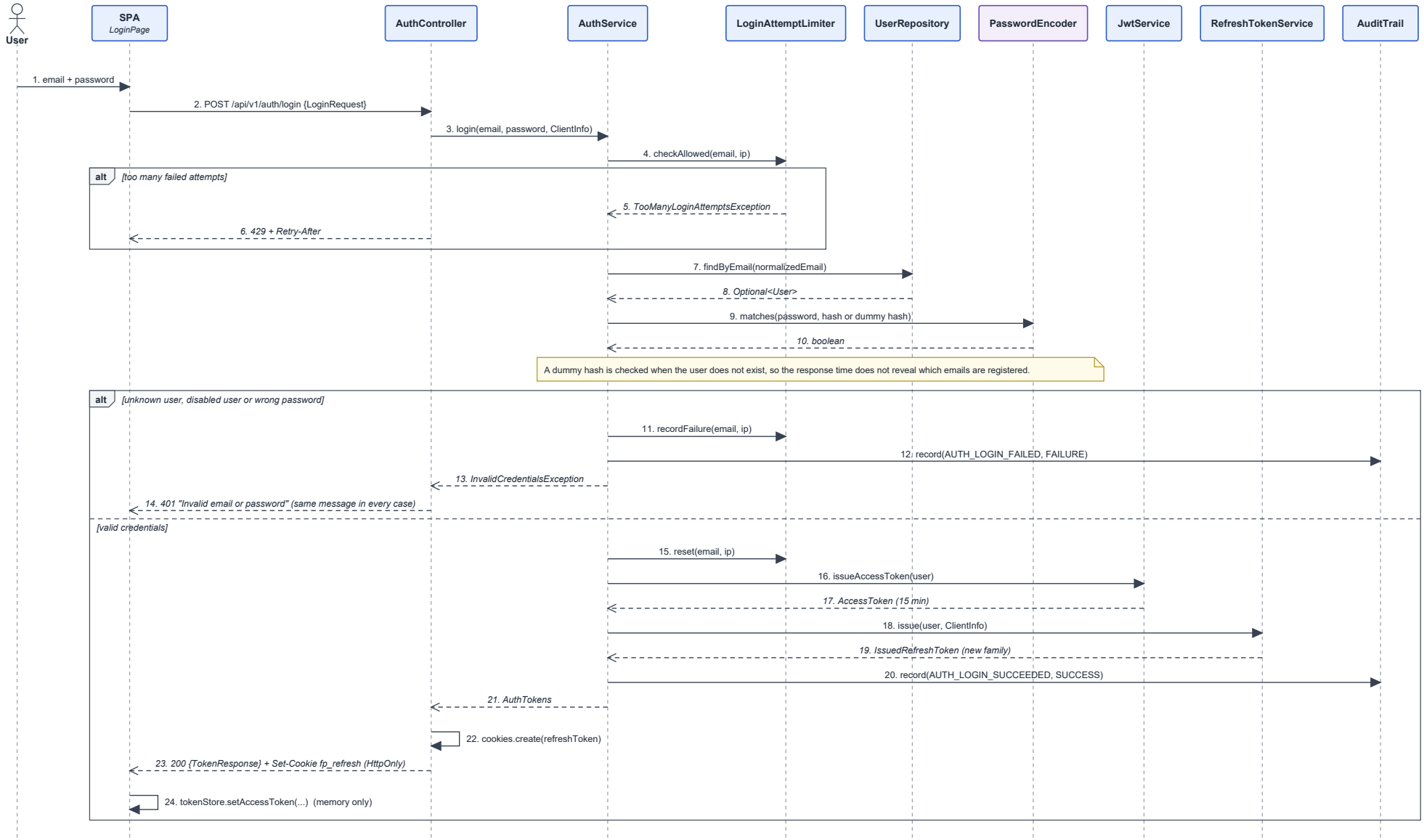
13 Sequence diagram: Register

Creating an account. Passwords are validated, hashed with Argon2id, and the event is audited.



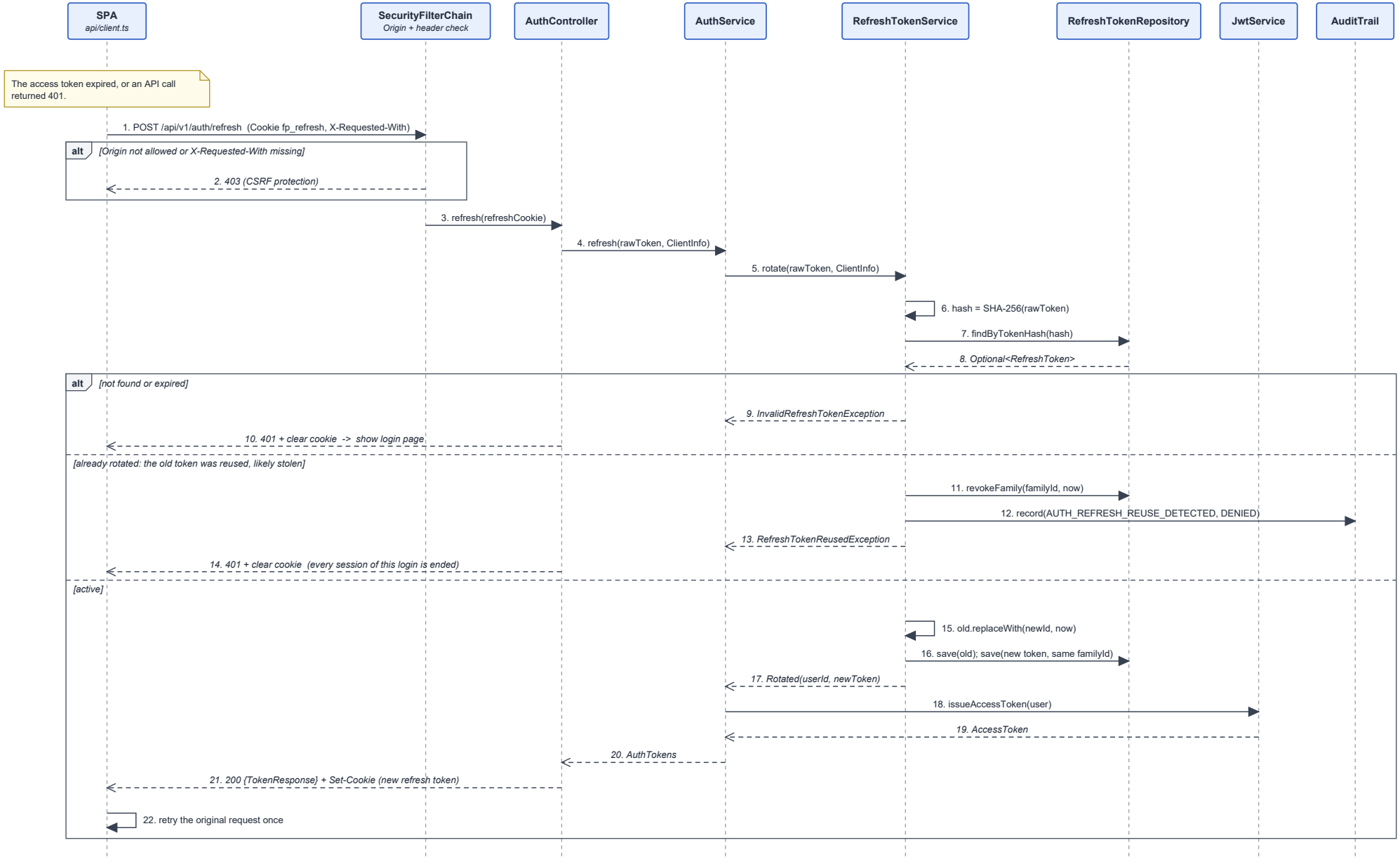
14 Sequence diagram: Log in

Rate-limited login that never reveals whether an email is registered. Issues a 15-minute access token and a refresh cookie.



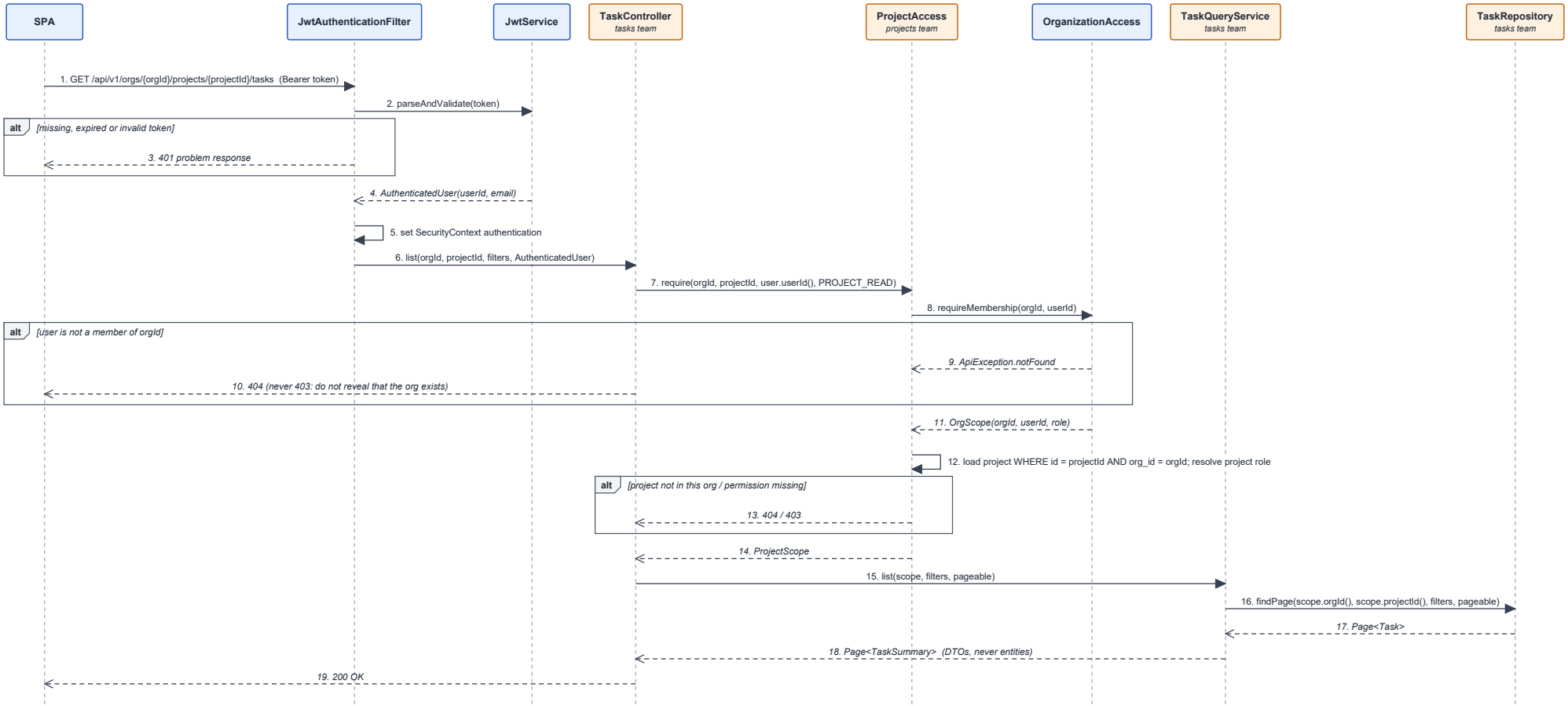
15 Sequence diagram: Refresh the session (rotation and reuse detection)

Every refresh replaces the refresh token. Presenting an already-used token ends the whole login family.



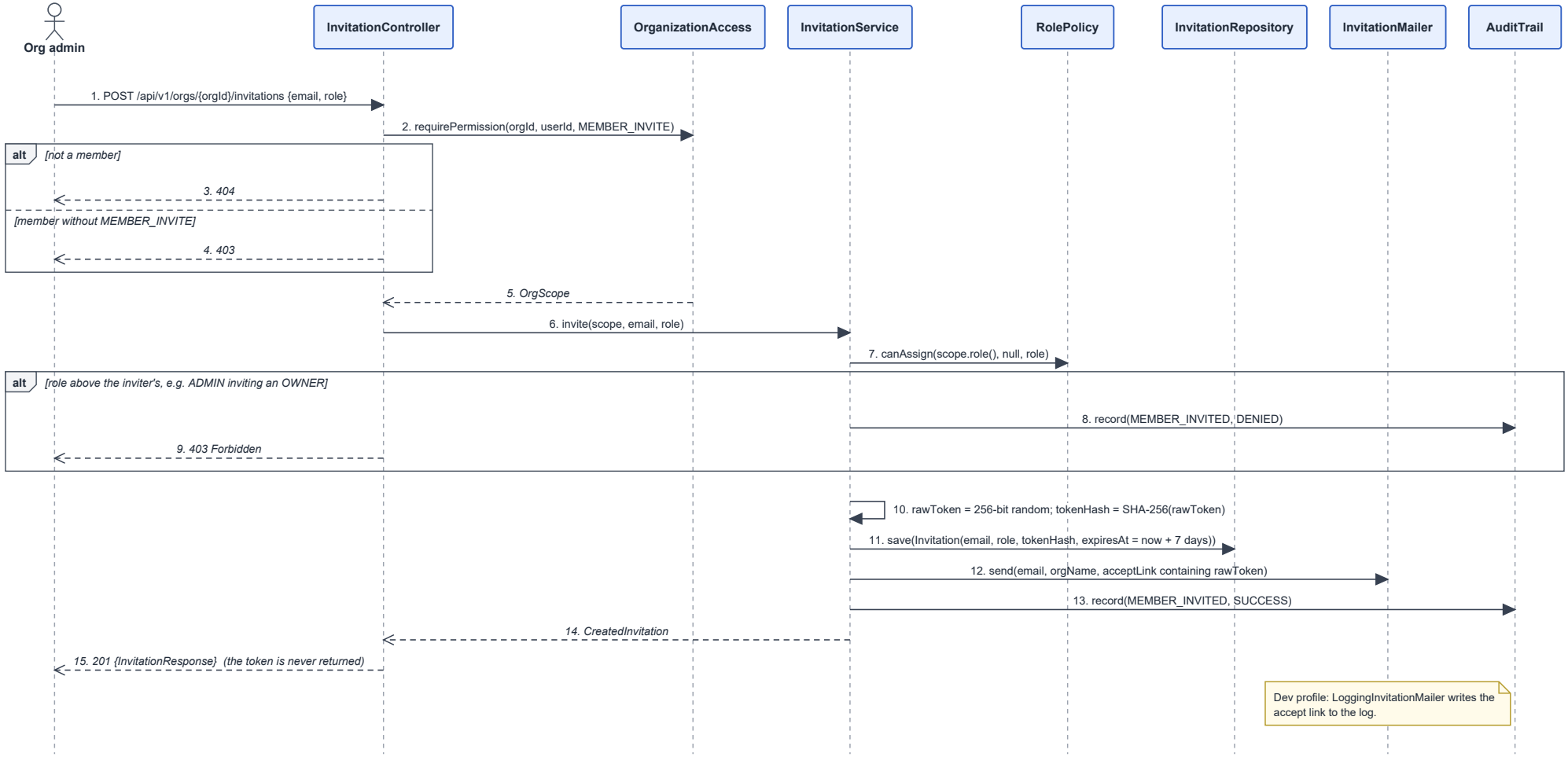
16 Sequence diagram: Authorized request: the pattern every module follows

Example with the tasks team's endpoint: authenticate, resolve ProjectScope through ProjectAccess and OrganizationAccess, then query with the scope.



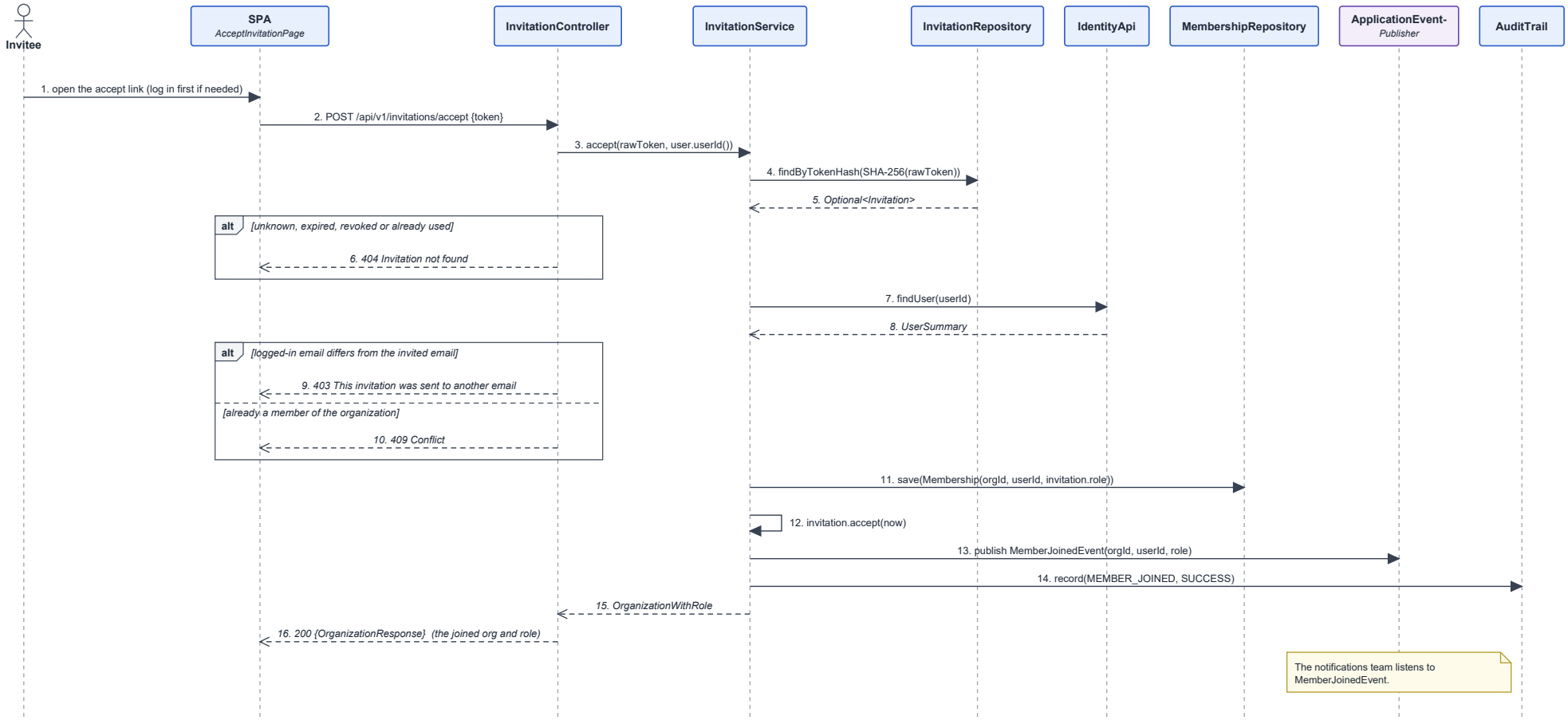
17 Sequence diagram: Invite a member

Only members with MEMBER_INVITE can invite, and never to a role above their own.



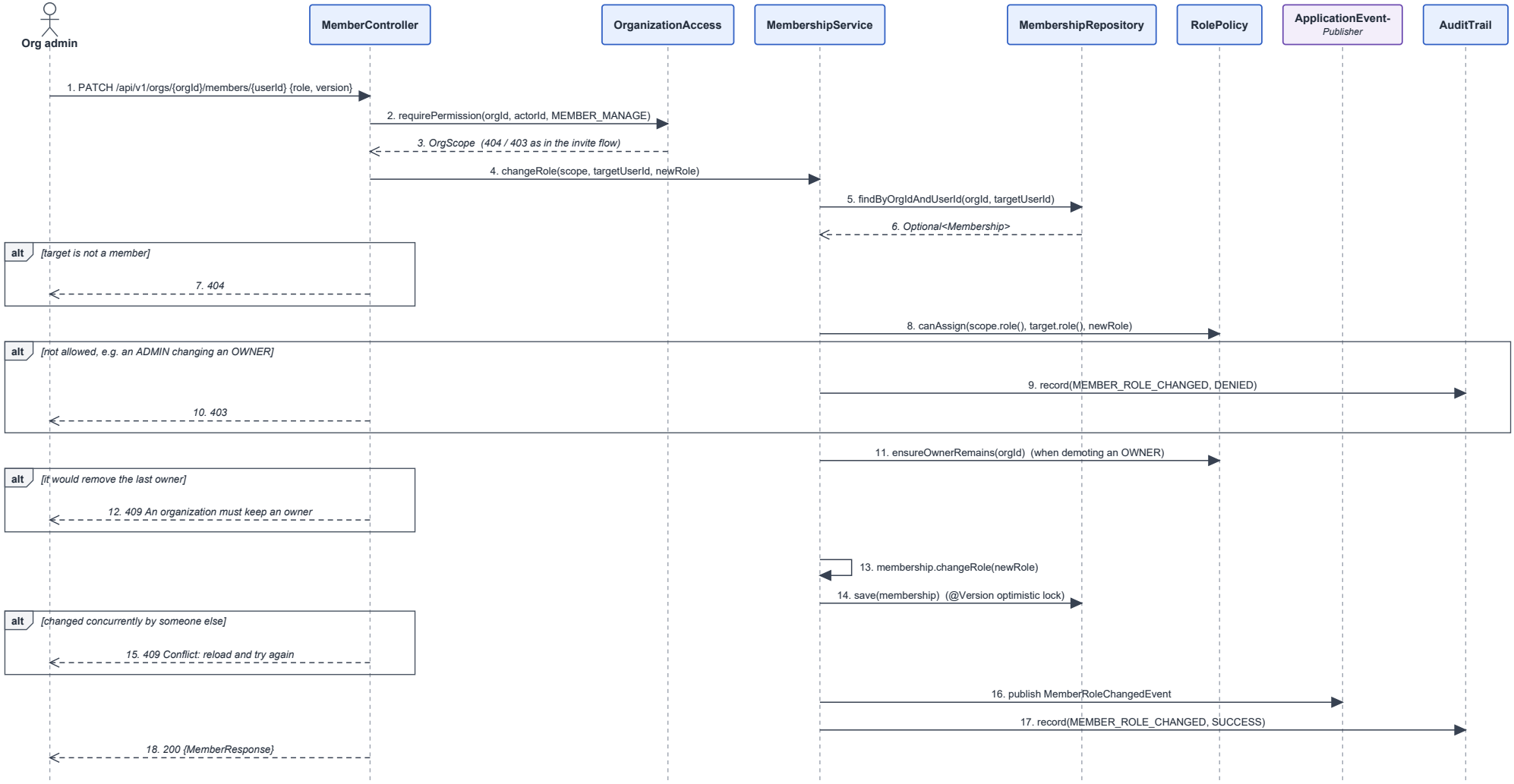
18 Sequence diagram: Accept an invitation

The invitation token is single-use, expires after 7 days and only works for the invited email.



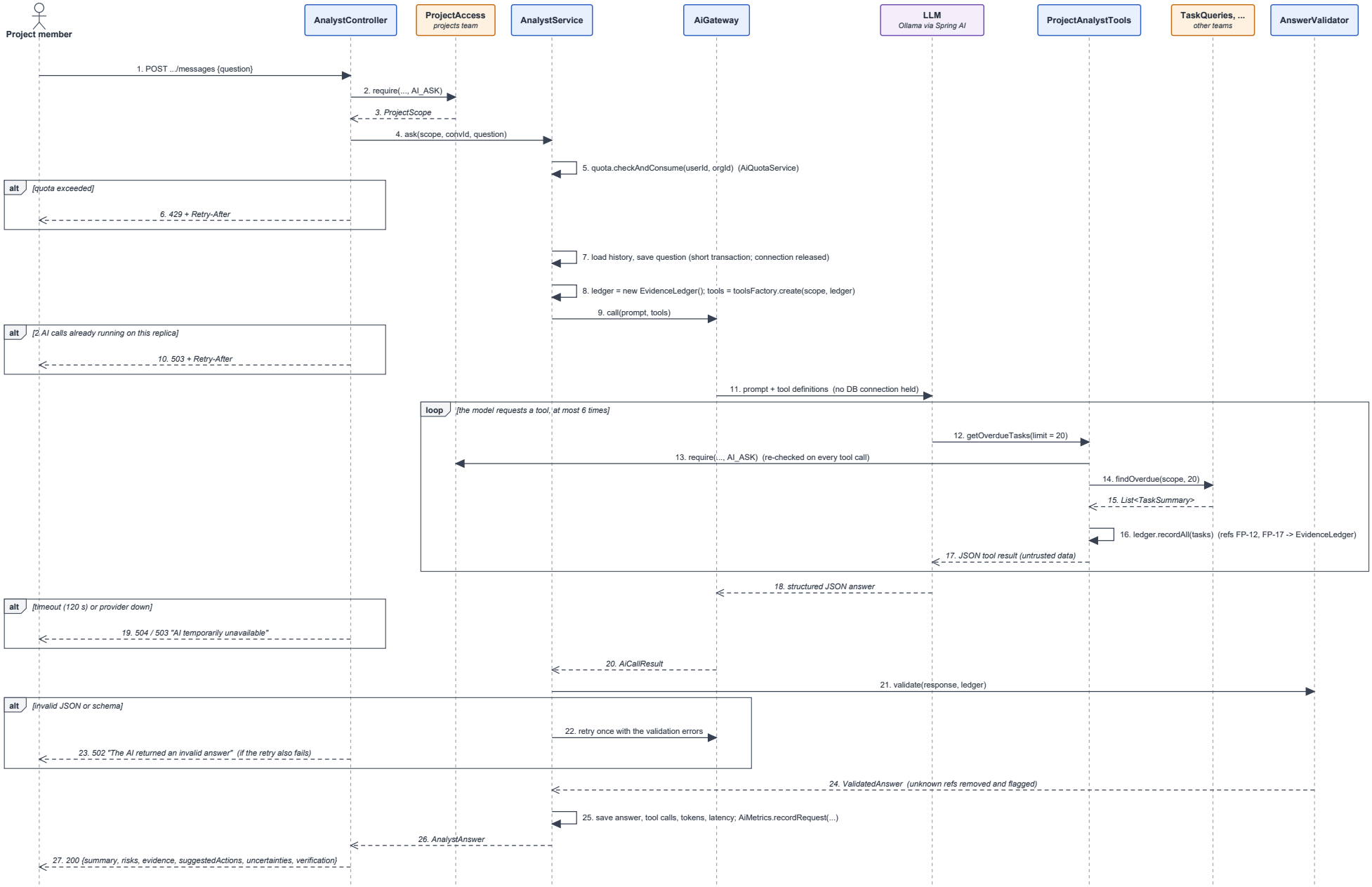
19 Sequence diagram: Change a member's role

Role rules, last-owner protection and optimistic locking.



20 Sequence diagram: Ask the AI Project Analyst

Authorization, quota, bounded concurrency, tool calling with evidence recording, and validation of the structured answer.



21 Roles and permissions

Organization roles map to organization permissions (OrgRole.permissions()). Project roles map to project permissions (implemented by the projects team). Org OWNER and ADMIN have the MANAGER project role on every project in the organization.

OrgPermission	OWNER	ADMIN	PROJECT_MANAGER	MEMBER	Used by
ORG_UPDATE	●	●	–	–	PATCH /orgs/{orgId}
ORG_DELETE	●	–	–	–	(later phase)
MEMBER_INVITE	●	●	–	–	invitations endpoints
MEMBER_MANAGE	●	● (not owners)	–	–	change role, remove member
PROJECT_CREATE	●	●	●	–	POST /orgs/{orgId}/projects (projects team)
PROJECT_MANAGE_ALL	●	●	–	–	implicit MANAGER on every project
AUDIT_VIEW	●	●	–	–	GET /orgs/{orgId}/audit

ProjectPermission	MANAGER	CONTRIBUTOR	VIEWER	Meaning
PROJECT_READ	●	●	●	see the project, tasks, milestones, analytics
PROJECT_EDIT	●	–	–	edit project details and milestones
MEMBER_MANAGE	●	–	–	add / remove project members (must be org members)
TASK_WRITE	●	●	–	create and edit tasks, comment
TASK_DELETE	●	● (own tasks)	–	delete tasks
DOCUMENT_WRITE	●	●	–	upload documents; contributors delete only their own
AI_ASK	●	●	●	use the AI analyst and the knowledge hub

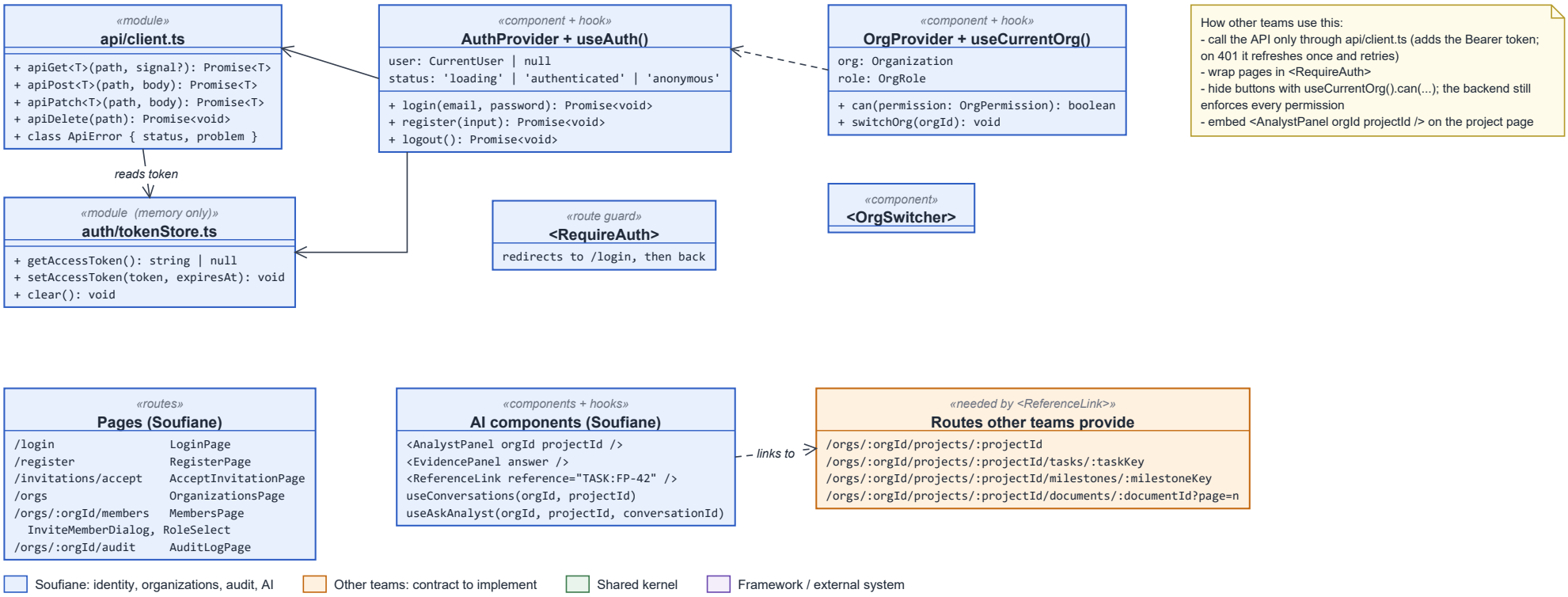
22 REST endpoints owned by Soufiane

Base path `/api/v1`. JSON bodies; errors are `application/problem+json`. `...` = `/api/v1/orgs/{orgId}/projects/{projectId}`.

Method	Path	Handler	Access
POST	`/api/v1/auth/register`	`AuthController.register`	public
POST	`/api/v1/auth/login`	`AuthController.login`	public, rate-limited
POST	`/api/v1/auth/refresh`	`AuthController.refresh`	refresh cookie + X-Requested-With
POST	`/api/v1/auth/logout`	`AuthController.logout`	refresh cookie
GET	`/api/v1/me`	`MeController.me`	authenticated
POST / GET	`/api/v1/orgs`	`OrganizationController.create / listMine`	authenticated
GET / PATCH	`/api/v1/orgs/{orgId}`	`OrganizationController.get / update`	member / ORG_UPDATE
GET	`/api/v1/orgs/{orgId}/members`	`MemberController.list`	member
PATCH / DELETE	`/api/v1/orgs/{orgId}/members/{userId}`	`MemberController.changeRole / remove`	MEMBER_MANAGE (members may remove themselves)
POST / GET	`/api/v1/orgs/{orgId}/invitations`	`InvitationController.invite / listPending`	MEMBER_INVITE
DELETE	`/api/v1/orgs/{orgId}/invitations/{invitationId}`	`InvitationController.revoke`	MEMBER_INVITE
POST	`/api/v1/invitations/accept`	`InvitationController.accept`	authenticated, invited email
GET	`/api/v1/orgs/{orgId}/audit`	`AuditController.search`	AUDIT_VIEW
POST / GET	`.../ai/conversations`	`AnalystController.startConversation / listConversations`	AI_ASK (own conversations only)
GET	`.../ai/conversations/{conversationId}`	`AnalystController.getConversation`	AI_ASK, owner only
POST	`.../ai/conversations/{conversationId}/messages`	`AnalystController.ask`	AI_ASK
POST	`.../ai/weekly-summary`	`AnalystController.weeklySummary`	AI_ASK

23 Frontend structure (Soufiane's parts and what other teams reuse)

Shared frontend building blocks: the API client, authentication, the current organization and the AI components. Other teams' pages reuse them instead of writing their own.



24 Open points for the team

- Confirm the query interface names and DTO fields in sections 11 and 12, or propose changes by pull request before Phase 3 starts.
- Projects team: confirm the human key format (`FP-42` for tasks, `FP-M3` for milestones) and that keys never change once assigned.
- Frontend: agree on the route paths in section 23, because the AI's clickable citations depend on them.
- Phase 4: the documents team owns `KnowledgeSearch`; the AI only calls it.